

LAPORAN PRATIKUM PEKAN 9
STRUKTUR DATA

Disusun Oleh:

Rifqi Aditya

2511533002

Dosen Pengampu :

Dr. Wahyudi, S.T, M.T

Asisten Praktikum :

Sherly Sukmadira



DEPARTEMEN INFORMATIKA
FAKULTAS TEKNOLOGI INFORMASI
UNIVERSITAS ANDALAS
2026

KATA PENGANTAR

Puji syukur ke hadirat Allah SWT karena atas rahmat dan karunia-Nya saya dapat menyelesaikan laporan praktikum mata Struktur Data tentang *Node*.

Laporan ini disusun untuk memenuhi salah satu tugas praktikum sekaligus sebagai sarana pembelajaran dalam memahami dasar-dasar pemrograman menggunakan bahasa Java. Melalui praktikum ini, penulis dapat mempelajari penggunaan *Node*.

Penulis menyadari bahwa laporan ini masih jauh dari sempurna. Oleh karena itu, penulis mengucapkan terima kasih kepada dosen pengampu, asisten praktikum, serta rekan-rekan yang telah membantu dalam proses penyusunan laporan ini.

Akhir kata, semoga laporan ini dapat memberikan manfaat, baik bagi penulis sendiri maupun bagi pembaca, khususnya dalam memperdalam pemahaman tentang pemrograman dasar dengan Java.

Padang, 11 Juni 2025

Penulis

Daftar Pustaka

BAB I PENDAHULUAN.....	4
1.1 Latar Belakang	4
1.2 Tujuan	4
1.3 Manfaat.....	4
BAB II PEMBAHASAN.....	5
2.1 Node.....	5
2.1.1 Kode Program	5
2.1.2 Uraian Kode Program.....	5
2.2 BTree	7
2.2.1 Kode Program	7
2.2.2 Uraian Kode Program.....	7
2.3 Btree Driver.....	9
2.3.1 Kode Program	9
2.3.2 Uraian Kode Program.....	9
2.3.3 Output	10
2.4 Graph Traversal	11
2.4.1 Kode Program	11
2.4.2 Uraian Kode Program.....	11
2.4.3 Output	13
BAB III TUGAS	13
3.1 Peta Lokasi.....	13
3.1.1 Kode Program	13
3.1.2 Output	15
3.1.3 Deskripsi Peta.....	16
3.1.4 Jumlah Node dan Edge	16
3.1.5 Hasil BFS	17
3.1.6 Hasil DFS	17
BAB III KESIMPULAN	18

BAB I

PENDAHULUAN

1.1 Latar Belakang

Pada praktikum ini, konsep tree diterapkan melalui Node, BTree, dan BTreeDriver. Node digunakan sebagai dasar pembentuk simpul, BTree digunakan untuk mengelola pohon biner, BTreeDriver digunakan untuk menjalankan dan menguji program tree yang telah dibuat.

Selain tree, praktikum ini juga membahas struktur data graph melalui program GraphTraversal. Graph digunakan untuk menggambarkan hubungan antar node yang saling terhubung, seperti peta lokasi, jaringan komputer, atau jalur perjalanan. Pada program GraphTraversal, dilakukan penelusuran menggunakan metode DFS (Depth First Search) dan BFS (Breadth First Search). Dengan adanya praktikum ini, mahasiswa dapat memahami perbedaan konsep tree dan graph serta cara melakukan penelusuran data pada kedua struktur tersebut.

1.2 Tujuan

1. Memahami konsep dasar Node sebagai simpul yang menyimpan data dan memiliki hubungan dengan simpul lainnya.
2. Mempelajari cara membuat dan mengelola struktur data Binary Tree menggunakan BTree dan menjalankannya melalui BTreeDriver.
3. Mengimplementasikan penelusuran pada struktur data Graph menggunakan algoritma DFS dan BFS pada program GraphTraversal.

1.3 Manfaat

1. Membantu mahasiswa memahami cara kerja struktur data tree dan graph dalam pemrograman Java,
2. Melatih kemampuan mahasiswa dalam membuat program berbasis class dan object, seperti Node, BTree, BTreeDriver, dan GraphTraversal.
3. Memberikan pemahaman tentang metode penelusuran data, baik pada binary tree menggunakan traversal Inorder, Preorder, Postorder, maupun pada graph menggunakan DFS dan BFS.

BAB II

PEMBAHASAN

2.1 Node

2.1.1 Kode Program

```
1 package Pekan9_2511533002;
2 public class Node_2511533002 {
3     int data_3002; // bisa diganti dengan string
4     Node_2511533002 left_3002;
5     Node_2511533002 right_3002;
6
7     public Node_2511533002 (int data_3002) {
8         this.data_3002 = data_3002;
9         left_3002 = null;
10        right_3002 = null;
11    }
12
13    public void setLeft_3002 (Node_2511533002 node_3002) {
14        if (left_3002 == null)
15            left_3002 = node_3002;
16    }
17
18    public void setRight_3002 (Node_2511533002 node_3002) {
19        if (right_3002 == null)
20            right_3002 = node_3002;
21    }
22
23    public Node_2511533002 getLeft_3002 () {
24        return left_3002;
25    }
26
27    public Node_2511533002 getRight_3002 () {
28        return right_3002;
29    }
30
31    public int getData_3002 () {
32        return data_3002;
33    }
34
35    public void setData_3002 (int data_3002) {
36        this.data_3002 = data_3002;
37    }
38
39    void printPreorder_3002 (Node_2511533002 node_3002) {
40        if (node_3002 == null)
41            return;
42        System.out.print(node_3002.data_3002 + " ");
43        printPreorder_3002 (node_3002.left_3002);
44        printPreorder_3002 (node_3002.right_3002);
45    }
46
47    void printPostorder_3002 (Node_2511533002 node_3002) {
48        if (node_3002 == null)
49            return;
50        printPostorder_3002 (node_3002.left_3002);
51        printPostorder_3002 (node_3002.right_3002);
52        System.out.print(node_3002.data_3002 + " ");
53    }
54
55    void printInorder_3002 (Node_2511533002 node_3002) {
56        if (node_3002 == null)
57            return;
58        printInorder_3002 (node_3002.left_3002);
59        System.out.print(node_3002.data_3002 + " ");
60        printInorder_3002 (node_3002.right_3002);
61    }
62
63    public String print_3002() {
64        return this.print_3002("", true, "");
65    }
66
67    public String print_3002(String prefix_3002, boolean isTail_3002, String sb_3002) {
68        if (right_3002 != null) {
69            right_3002.print_3002(prefix_3002 + (isTail_3002 ? " | " : " "), false, sb_3002);
70        }
71        System.out.println(prefix_3002 + (isTail_3002 ? " | " : " ") + data_3002);
72        if (left_3002 != null) {
73            left_3002.print_3002(prefix_3002 + (isTail_3002 ? " | " : " "), true, sb_3002);
74        }
75        return sb_3002;
76    }
77 }
```

2.1.2 Uraian Kode Program

1. **Node_2511533002 left_3002;** = Variabel untuk menyimpan alamat node sebelah kiri.
2. **Node_2511533002 right_3002;** = Variabel untuk menyimpan alamat node anak sebelah kanan.
3. **left_3002 = null;** = Mengatur kiri node menjadi kosong saat node pertama kali dibuat.
4. **right_3002 = null;** = Mengatur kanan node menjadi kosong saat node pertama kali dibuat.
5. **left_3002 = node_3002;** = Jika kiri masih kosong, maka node baru akan dimasukkan sebagai anak kiri.

6. **right_3002 = node_3002;** = Jika kanan masih kosong, maka node baru akan dimasukkan sebagai anak kanan.
7. **public void setData_3002(int data_3002) {** = Method untuk mengubah nilai data pada node.
8. **this.data_3002 = data_3002;** = Mengganti nilai data lama dengan nilai data baru yang dikirim melalui parameter.
9. **void printPreorder_3002(Node_2511533002 node_3002) {** = Method untuk menampilkan data tree dengan urutan Preorder.
10. **printPreorder_3002(node_3002.left_3002);** = Melanjutkan traversal ke kiri.
11. **printPreorder_3002(node_3002.right_3002);** = Melanjutkan traversal ke kanan.
12. **public String print_3002() {** = Method untuk memanggil proses pencetakan bentuk tree secara sederhana tanpa harus mengisi parameter.
13. **return this.print_3002("", true, "");** = Memanggil method **print_3002** lain yang memiliki parameter. Nilai awal prefix kosong, posisi dianggap sebagai tail, dan string awal kosong
14. **public String print_3002(String prefix_3002, boolean isTail_3002, String sb_3002) {** = Method untuk menampilkan bentuk tree secara visual menggunakan simbol garis seperti `--` dan `/--`
15. **right_3002.print_3002(prefix_3002 + (isTail_3002 ? "| " : " "), false, sb_3002);**
= Jika anak kanan ada, maka anak kanan akan dicetak terlebih dahulu. Prefix digunakan untuk mengatur jarak atau garis agar bentuk tree terlihat rapi.
16. **System.out.println(prefix_3002 + (isTail_3002 ? "\\--" : "/--") + data_3002);** = Menampilkan data node saat ini dengan tambahan simbol garis. Jika `isTail_3002` bernilai `true`, maka menggunakan simbol `--`. Jika `isTail_3002` bernilai `false`, maka menggunakan simbol `/--`.
17. **left_3002.print_3002(prefix_3002 + (isTail_3002 ? " " : "| "), true, sb_3002);**
= Jika anak kiri ada, maka anak kiri akan dicetak setelah node utama dan anak kanan.

2.2 BTree

2.2.1 Kode Program

```
1 package Pekan2_2511533002;
2 public class BTree_2511533002 {
3     private Node_2511533002 root_3002;
4     private Node_2511533002 currentNode_3002;
5     public BTree_2511533002 () {
6         root_3002 = null;
7     }
8
9     public boolean search_3002 (int data_3002) {
10        return search_3002 (root_3002, data_3002);
11    }
12
13    private boolean search_3002 (Node_2511533002 node_3002 , int data_3002) {
14        if (node_3002.getData_3002() == data_3002) {
15            return true;
16        }
17        if (node_3002.getLeft_3002() != null) {
18            if (search_3002(node_3002.getLeft_3002(), data_3002))
19                return true;
20        }
21        if (node_3002.getRight_3002() != null) {
22            if (search_3002(node_3002.getRight_3002(), data_3002))
23                return true;
24        }
25        return false;
26    }
27
28    public void printInorder_3002 () {
29        root_3002.printInorder_3002(root_3002);
30    }
31
32    public void printPreorder_3002 () {
33        root_3002.printPreorder_3002(root_3002);
34    }
35
36    public void printPostorder_3002 () {
37        root_3002.printPostorder_3002(root_3002);
38    }
39
40    public Node_2511533002 getRoot_3002 () {
41        return root_3002;
42    }
43
44    public boolean isEmpty_3002 () {
45        return root_3002 == null;
46    }
47
48    public int countNodes_3002 () {
49        return countNodes_3002 (root_3002);
50    }
51
52    private int countNodes_3002 (Node_2511533002 node_3002) {
53        int count_3002 = 1;
```

```
54        if (node_3002 == null){
55            return 0;
56        }
57        else {
58            count_3002 += countNodes_3002 (node_3002.getLeft_3002());
59            count_3002 += countNodes_3002 (node_3002.getRight_3002());
60            return count_3002;
61        }
62    }
63
64    public void print_3002 () {
65        root_3002.print_3002();
66    }
67
68    public Node_2511533002 getCurrent_3002 () {
69        return currentNode_3002;
70    }
71
72    public Node_2511533002 setCurrent_3002 (Node_2511533002 node_3002) {
73        return currentNode_3002 = node_3002;
74    }
75
76    public void setRoot_3002 (Node_2511533002 root_3002) {
77        this.root_3002 = root_3002;
78    }
79 }
```

2.2.2 Uraian Kode Program

1. **public boolean search_3002(int data_3002)** { = Method public untuk mencari data tertentu di dalam binary tree.
2. **return search_3002(root_3002, data_3002);** = Memanggil method search private dengan memulai pencarian dari node root.
3. **if (node_3002.getData_3002() == data_3002)** = Mengecek apakah data pada node saat ini sama dengan data yang dicari.
4. **if (node_3002.getLeft_3002() != null)** = Mengecek apakah node saat ini memiliki ada di kiri.
5. **if (search_3002(node_3002.getLeft_3002(), data_3002))** = Jika di kiri ada, maka pencarian dilanjutkan ke bagian kiri tree.
6. **if (node_3002.getRight_3002() != null)** = Mengecek apakah node saat ini memiliki ada di kanan.
7. **if (search_3002(node_3002.getRight_3002(), data_3002))** = Jika noda kanan ada, maka pencarian dilanjutkan ke bagian kanan tree secara rekursif.
8. **if (search_3002(node_3002.getRight_3002(), data_3002))** = Jika anak kanan ada, maka pencarian dilanjutkan ke bagian kanan tree.

9. **root_3002.printInorder_3002(root_3002);** = Memanggil method `printInorder_3002` dari class `Node_2511533002` dengan memulai dari root.
10. **root_3002.printPreorder_3002(root_3002);** = Memanggil method `printPreorder_3002` dari class `Node_2511533002` dengan memulai dari root.
11. **root_3002.printPostorder_3002(root_3002);** = Memanggil method `printPostorder_3002` dari class `Node_2511533002` dengan memulai dari root.
12. **public boolean isEmpty_3002()** { = Method untuk mengecek apakah binary tree masih kosong atau sudah memiliki node.
13. **public int countNodes_3002()** { = Method public untuk menghitung jumlah seluruh node yang ada di dalam binary tree.
14. **int count_3002 = 1;** = Membuat variabel `count_3002` dengan nilai awal 1. Nilai ini mewakili node yang sedang dihitung.
15. **if (node_3002 == null)** { = Mengecek apakah node kosong.
16. **count_3002 += countNodes_3002(node_3002.getLeft_3002());** = Menambahkan jumlah node dari kiri ke variabel `count_3002`.
17. **count_3002 += countNodes_3002(node_3002.getRight_3002());** = Menambahkan jumlah node dari kanan ke variabel `count_3002`.
18. **public Node_2511533002 getCurrent_3002()** { = Method untuk mengambil node yang sedang aktif atau sedang digunakan.
19. **public Node_2511533002 setCurrent_3002(Node_2511533002 node_3002)** { = Method untuk mengatur node yang sedang aktif.
20. **return currentNode_3002 = node_3002;** = Mengisi `currentNode_3002` dengan node yang dikirim melalui parameter, lalu mengembalikannya.
21. **public void setRoot_3002(Node_2511533002 root_3002)** {
= Method untuk mengatur atau mengganti root dari binary tree.

2.3 Btree Driver

2.3.1 Kode Program

```
1 package Peko9_2511533002;
2 public class BTreeDriver_2511533002 {
3     public static void main (String [] args) {
4         //membuat pohon
5         BTree_2511533002 tree_3002 = new BTree_2511533002 ();
6         System.out.print("Jumlah Simpul Awal Pohon : ");
7         System.out.println(tree_3002.countNodes_3002());
8
9         //menambahkan simpul data 1
10        Node_2511533002 root_3002 = new Node_2511533002 (1);
11
12        //menjadikan simpul 1 sebagai root
13        tree_3002.setRoot_3002(root_3002);
14        System.out.print("Jumlah simpul jika hanya ada root :");
15        System.out.println(tree_3002.countNodes_3002());
16        Node_2511533002 node2_3002 = new Node_2511533002 (2);
17        Node_2511533002 node3_3002 = new Node_2511533002 (3);
18        Node_2511533002 node4_3002 = new Node_2511533002 (4);
19        Node_2511533002 node5_3002 = new Node_2511533002 (5);
20        Node_2511533002 node6_3002 = new Node_2511533002 (6);
21        Node_2511533002 node7_3002 = new Node_2511533002 (7);
22        Node_2511533002 node8_3002 = new Node_2511533002 (8);
23        Node_2511533002 node9_3002 = new Node_2511533002 (9);
24
25        root_3002.setLeft_3002(node2_3002);
26        node2_3002.setLeft_3002(node4_3002);
27        node2_3002.setRight_3002(node5_3002);
28        node4_3002.setRight_3002(node8_3002);
29        root_3002.setRight_3002(node3_3002);
30        node3_3002.setLeft_3002(node6_3002);
31        node3_3002.setRight_3002(node7_3002);
32        node6_3002.setLeft_3002(node9_3002);
33
34        //Set root
35        tree_3002.setCurrent_3002(tree_3002.getRoot_3002());
36        System.out.println("Menampilkan simpul terakhir: ");
37        System.out.println(tree_3002.getCurrent_3002().getData_3002());
38        System.out.println("Jumlah simpul: setelah simpul 7 ditambahkan");
39        System.out.println(tree_3002.countNodes_3002());
40        System.out.println("InOrder: ");
41        tree_3002.printInorder_3002();
42        System.out.println("PreOrder: ");
43        tree_3002.printPreorder_3002();
44        System.out.println("PostOrder: ");
45        tree_3002.printPostorder_3002();
46        System.out.println("\nMenampilkan simpul dalam bentuk pohon");
47        tree_3002.print_3002();
48    }
49 }
```

2.3.2 Uraian Kode Program

1. **System.out.print("Jumlah Simpul Awal Pohon : ");** = Menampilkan teks untuk memberi keterangan jumlah simpul awal pada pohon.
2. **System.out.println(tree_3002.countNodes_3002());** = Menampilkan jumlah node awal pada tree. Karena tree masih kosong, hasilnya adalah 0.
3. **Node_2511533002 root_3002 = new Node_2511533002(1);** = Membuat node baru dengan nilai 1. Node ini akan dijadikan sebagai root atau akar pohon.
4. **tree_3002.setRoot_3002(root_3002);**
= Menjadikan node bernilai 1 sebagai root dari binary tree.
5. **System.out.print("Jumlah simpul jika hanya ada root :");** = Menampilkan teks keterangan bahwa tree baru memiliki root saja.
6. **System.out.println(tree_3002.countNodes_3002());** = Menghitung jumlah node setelah root dimasukkan. Karena baru ada satu node, hasilnya adalah 1.
7. **Node_2511533002 node2_3002 = new Node_2511533002(2);** = Membuat node baru dengan nilai 2.
8. **Node_2511533002 node3_3002 = new Node_2511533002(3);** = Membuat node baru dengan nilai 3.
9. **Node_2511533002 node4_3002 = new Node_2511533002(4);** = Membuat node baru dengan nilai 4.
10. **Node_2511533002 node5_3002 = new Node_2511533002(5);** = Membuat node baru dengan nilai 5.
11. **Node_2511533002 node6_3002 = new Node_2511533002(6);** = Membuat node baru dengan nilai 6.

12. **Node_2511533002 node7_3002 = new Node_2511533002(7);** = Membuat node baru dengan nilai 7.
13. **Node_2511533002 node8_3002 = new Node_2511533002(8);** = Membuat node baru dengan nilai 8.
14. **Node_2511533002 node9_3002 = new Node_2511533002(9);** = Membuat node baru dengan nilai 9.
15. **root_3002.setLeft_3002(node2_3002);** = Menjadikan node 2 sebagai anak kiri dari root 1.
16. **node2_3002.setLeft_3002(node4_3002);** = Menjadikan node 4 sebagai kiri dari node 2.
17. **node2_3002.setRight_3002(node5_3002);** = Menjadikan node 5 sebagai kanan dari node 2.
18. **node4_3002.setRight_3002(node8_3002);** = Menjadikan node 8 sebagai kanan dari node 4
19. **root_3002.setRight_3002(node3_3002);** = Menjadikan node 3 sebagai kanan dari root 1.
20. **node3_3002.setLeft_3002(node6_3002);** = Menjadikan node 6 sebagai kiri dari node 3.
21. **node3_3002.setRight_3002(node7_3002);** = Menjadikan node 7 sebagai kanan dari node 3.
22. **node6_3002.setLeft_3002(node9_3002);** = Menjadikan node 9 sebagai kiri dari node 6.
23. **tree_3002.setCurrent_3002(tree_3002.getRoot_3002());** = Mengatur currentNode_3002 agar menunjuk ke root dari tree, yaitu node bernilai 1.
24. **System.out.println(tree_3002.getCurrent_3002().getData_3002());** = Menampilkan data dari current node. Karena current node diatur ke root, maka yang tampil Adalah 1.

2.3.3 Output

```
Jumlah Simpul Awal Pohon : 0
Jumlah simpul jika hanya ada root :1
menampilkan simpul terakhir:
1
Jumlah simpul; setelah simpul 7 ditambahkan
9
InOrder:
4 8 2 5 1 9 6 3 7
PreOrder:
1 2 4 8 5 3 6 9 7
PostOrder:
8 4 5 2 9 6 7 3 1
Menampilkan simpul dalam bentuk pohon
|
|--7
|
|--3
|   |--6
|   |--9
|--1
|   |--5
|   |--2
|       |--8
|       |--4
```

2.4 Graph Traversal

2.4.1 Kode Program

```
1 package Pekan2_2511533002;
2 import java.util.*;
3 public class GraphTraversal_2511533002 {
4     private Map<String, List<String>> graph_3002 = new HashMap<>();
5
6     //Menambahkan edge (graf tak berarah)
7     public void addEdge_3002 (String node1_3002, String node2_3002) {
8         graph_3002.putIfAbsent(node1_3002, new ArrayList<>());
9         graph_3002.putIfAbsent(node2_3002, new ArrayList<>());
10        graph_3002.get(node1_3002).add(node2_3002);
11        graph_3002.get(node2_3002).add(node1_3002);
12    }
13
14    //menampilkan graf awal
15    public void printGraph_3002() {
16        System.out.println("Graf Awal (Adjacency List) : ");
17        for (String node_3002 : graph_3002.keySet()) {
18            System.out.print(node_3002 + "-> ");
19            List<String> neighbors_3002 = graph_3002.get(node_3002);
20            System.out.println(String.join(", ", neighbors_3002));
21        }
22        System.out.println();
23    }
24
25    //DFS rekursif
26    public void dfs_3002 (String start_3002) {
27        Set<String> visited_3002 = new HashSet<>();
28        System.out.println("Penelusuran DFS : ");
29        dfsHelper_3002(start_3002, visited_3002);
30        System.out.println();
31    }
32
33    private void dfsHelper_3002 (String current_3002, Set<String> visited_3002) {
34        if (visited_3002.contains(current_3002)) return;
35        visited_3002.add(current_3002);
36        System.out.print(current_3002 + " ");
37        for (String neighbor_3002 : graph_3002.getOrDefault(current_3002, new ArrayList<>())) {
38            dfsHelper_3002(neighbor_3002, visited_3002);
39        }
40    }
41
42    private void bfs_3002 (String start_3002) {
43        Set<String> visited_3002 = new HashSet<>();
44        Queue<String> queue_3002 = new LinkedList<>();
45        queue_3002.add(start_3002);
46        visited_3002.add(start_3002);
47        System.out.println("Penelusuran BFS : ");
48        while (!queue_3002.isEmpty()) {
49            String current_3002 = queue_3002.poll();
50            System.out.print(current_3002 + " ");
51
52            for (String neighbor_3002 : graph_3002.getOrDefault(current_3002, new ArrayList<>())) {
53                if (!visited_3002.contains(neighbor_3002)) {
54                    queue_3002.add(neighbor_3002);
55                    visited_3002.add(neighbor_3002);
56                }
57            }
58            System.out.println();
59        }
60    }
61    //main
62    public static void main (String [] Args) {
63        GraphTraversal_2511533002 graph_3002 = new GraphTraversal_2511533002();
64
65        //contoh graf = A-B, A-C, B-D, B-E
66        graph_3002.addEdge_3002("A", "B");
67        graph_3002.addEdge_3002("A", "C");
68        graph_3002.addEdge_3002("B", "D");
69        graph_3002.addEdge_3002("B", "E");
70
71        //tampilkan graf awal
72        System.out.println("Graf Awal adalah : ");
73        graph_3002.printGraph_3002();
74
75        //lakukan penelusuran
76        graph_3002.dfs_3002("A");
77        graph_3002.bfs_3002("A");
78    }
79 }
```

2.4.2 Uraian Kode Program

1. **private Map<String, List<String>> graph_3002 = new HashMap<>();** = Membuat variabel graph_3002 untuk menyimpan graf. Tipe Map<String, List<String>> berarti setiap node berupa String, dan setiap node memiliki daftar tetangga berupa List<String>.
2. **public void addEdge_3002(String node1_3002, String node2_3002) {** = Method untuk menambahkan hubungan atau edge antara dua node dalam graf.
3. **graph_3002.putIfAbsent(node1_3002, new ArrayList<>());** = Jika node1_3002 belum ada di dalam graph, maka node tersebut akan ditambahkan dengan daftar tetangga kosong.
4. **graph_3002.putIfAbsent(node2_3002, new ArrayList<>());** = Jika node2_3002 belum ada di dalam graph, maka node tersebut juga akan ditambahkan dengan daftar tetangga kosong.
5. **graph_3002.get(node1_3002).add(node2_3002);** = Menambahkan node2_3002 sebagai tetangga dari node1_3002.

6. **graph_3002.get(node2_3002).add(node1_3002);** = Menambahkan node1_3002 sebagai tetangga dari node2_3002.
7. **List<String> neighbors_3002 = graph_3002.get(node_3002);** = Mengambil daftar tetangga dari node yang sedang diproses.
8. **System.out.println(String.join(",", neighbors_3002));** = Menampilkan semua tetangga dari node tersebut, lalu dipisahkan dengan tanda koma.
9. **Set<String> visited_3002 = new HashSet<>();** = Membuat kumpulan data bernama visited_3002 untuk menyimpan node yang sudah dikunjungi.
10. **dfsHelper_3002(start_3002, visited_3002);** = Memanggil method bantuan dfsHelper_3002 untuk menjalankan DFS mulai dari node awal.
11. **if (visited_3002.contains(current_3002)) return;** = Mengecek apakah node saat ini sudah pernah dikunjungi. Jika sudah, maka proses dihentikan agar tidak terjadi pengulangan
12. **visited_3002.add(current_3002);** = Menandai node saat ini sebagai node yang sudah dikunjungi
13. **for (String neighbor_3002 : graph_3002.getDefault(current_3002, new ArrayList<>())) {** = Melakukan perulangan untuk mengambil semua tetangga dari node saat ini. Jika node tidak memiliki tetangga, maka akan menggunakan list kosong.
14. **Set<String> visited_3002 = new HashSet<>();** = Membuat kumpulan data untuk menyimpan node yang sudah dikunjungi.
15. **Queue<String> queue_3002 = new LinkedList<>();** = Membuat antrian menggunakan Queue. Queue digunakan dalam BFS karena BFS bekerja dengan prinsip FIFO atau First In First Out.
16. **queue_3002.add(start_3002);** = Memasukkan node awal ke dalam antrian.
17. **visited_3002.add(start_3002);** = Menandai node awal sebagai node yang sudah dikunjungi.
18. **String current_3002 = queue_3002.poll();** = Mengambil dan menghapus node paling depan dari antrian untuk diproses.
19. **for (String neighbor_3002 : graph_3002.getDefault(current_3002, new ArrayList<>())) {** = Melakukan perulangan untuk mengambil semua tetangga dari node saat ini.
20. **if (!visited_3002.contains(neighbor_3002)) {** = Mengecek apakah tetangga tersebut belum pernah dikunjungi.
21. **queue_3002.add(neighbor_3002);** = Jika belum dikunjungi, maka tetangga tersebut dimasukkan ke dalam antrian.
22. **visited_3002.add(neighbor_3002);** = Menandai tetangga tersebut sebagai node yang sudah dikunjungi.
23. **graph_3002.addEdge_3002("A", "C");** = Menambahkan hubungan antara node A dan C.
24. **graph_3002.addEdge_3002("B", "D");** = Menambahkan hubungan antara node B dan D.
25. **graph_3002.addEdge_3002("B", "E");** = Menambahkan hubungan antara node B dan E.

2.4.3 Output

```
Graf Awal adalah :  
Graf Awal (Adjacency List) :  
A-> B,C  
B-> A,D,E  
C-> A  
D-> B  
E-> B  
  
Penelusuran DFS :  
A B D E C  
Penelusuran BFS :  
A B C D E
```

BAB III TUGAS

3.1 Peta Lokasi

3.1.1 Kode Program

```
1 package Pekan9_2511533002;  
2  
3 import javax.swing.*;  
4 import java.awt.*;  
5 import java.util.*;  
6  
7 public class Petalokasi_2511533002 extends JFrame {  
8  
9     private static final long serialVersionUID = 1L;  
10  
11     private Map<String, ArrayList<String>> graph_3002 = new LinkedHashMap<>();  
12     private JComboBox<String> comboAwal_3002;  
13     private JComboBox<String> comboTujuan_3002;  
14     private JTextArea areaGraph_3002;  
15     private JTextArea areaHasil_3002;  
16  
17     public Petalokasi_2511533002() {}  
18     buatGraph_3002();  
19     buatGUI_3002();  
20     displayGraph();  
21  
22  
23     private void buatGraph_3002() {  
24         tambahEdge_3002("Gerbang", "Parkiran");  
25         tambahEdge_3002("Gerbang", "Rektorat");  
26         tambahEdge_3002("Parkiran", "Kantin");  
27         tambahEdge_3002("Parkiran", "Perpus");  
28         tambahEdge_3002("Rektorat", "Perpus");  
29         tambahEdge_3002("Rektorat", "Aula");  
30         tambahEdge_3002("Perpus", "Lab Komputer");  
31         tambahEdge_3002("Perpus", "FTI");  
32         tambahEdge_3002("Kantin", "Asrama");  
33         tambahEdge_3002("Kantin", "Lab Komputer");  
34         tambahEdge_3002("Lab Komputer", "Klinik");  
35         tambahEdge_3002("Lab Komputer", "Masjid");  
36         tambahEdge_3002("FTI", "Masjid");  
37         tambahEdge_3002("Aula", "FTI");  
38         tambahEdge_3002("Asrama", "Klinik");  
39         tambahEdge_3002("Masjid", "Klinik");  
40     }  
41  
42     private void tambahEdge_3002(String node1_3002, String node2_3002) {  
43         graph_3002.putIfAbsent(node1_3002, new ArrayList<>());  
44         graph_3002.putIfAbsent(node2_3002, new ArrayList<>());
```

```

46     graph_3002.get(node1_3002).add(node2_3002);
47     graph_3002.get(node2_3002).add(node1_3002);
48 }
49
50 private void buatGUI_3002() {
51     setTitle("Pencarian Jalur BFS dan DFS - 2511533002");
52     setSize(750, 520);
53     setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
54     setLocationRelativeTo(null);
55     setLayout(new BorderLayout());
56
57     JLabel judul_3002 = new JLabel("PEMKARIAN JALUR PETA KAMPUS", JLabel.CENTER);
58     judul_3002.setFont(new Font("Arial", Font.BOLD, 18));
59     judul_3002.setOpaque(true);
60     judul_3002.setBackground(new Color(40, 80, 130));
61     judul_3002.setForeground(Color.WHITE);
62     judul_3002.setPreferredSize(new Dimension(750, 45));
63     add(judul_3002, BorderLayout.NORTH);
64
65     JPanel panelUtama_3002 = new JPanel(new BorderLayout(10, 10));
66     panelUtama_3002.setBorder(BorderFactory.createEmptyBorder(10, 10, 10, 10));
67     add(panelUtama_3002, BorderLayout.CENTER);
68
69     JPanel panelInput_3002 = new JPanel(new FlowLayout(FlowLayout.LEFT));
70
71     JComboBox_3002 = new JComboBox<>(graph_3002.keySet().toArray(new String[0]));
72     JComboBox_tujuan_3002 = new JComboBox<>(graph_3002.keySet().toArray(new String[0]));
73     JComboBox_tujuan_3002.setSelectedItem("Klinik");
74
75     JButton tombolBFS_3002 = new JButton("[ BFS ]");
76     JButton tombolDFS_3002 = new JButton("[ DFS ]");
77     JButton tombolReset_3002 = new JButton("[ RESET ]");
78
79     tombolBFS_3002.addActionListener(e -> BFS());
80     tombolDFS_3002.addActionListener(e -> DFS());
81     tombolReset_3002.addActionListener(e -> resetGraph());
82
83     panelInput_3002.add(new JLabel("Lokasi Awal :"));
84     panelInput_3002.add(comboAwal_3002);
85     panelInput_3002.add(new JLabel("Lokasi Tujuan :"));
86     panelInput_3002.add(comboTujuan_3002);
87     panelInput_3002.add(tombolBFS_3002);
88     panelInput_3002.add(tombolDFS_3002);
89     panelInput_3002.add(tombolReset_3002);

```

```

91     panelUtama_3002.add(panelInput_3002, BorderLayout.NORTH);
92
93     areaGraph_3002 = new JTextArea();
94     areaGraph_3002.setEditable(false);
95     areaGraph_3002.setFont(new Font("Monospaced", Font.BOLD, 14));
96     areaGraph_3002.setBorder(BorderFactory.createTitledBorder("VISUALISASI GRAPH"));
97
98     panelUtama_3002.add(areaGraph_3002, BorderLayout.CENTER);
99
100     areaHasil_3002 = new JTextArea();
101     areaHasil_3002.setEditable(false);
102     areaHasil_3002.setFont(new Font("Monospaced", Font.BOLD, 13));
103     areaHasil_3002.setBorder(BorderFactory.createTitledBorder("HASIL PENCARIAN"));
104     areaHasil_3002.setPreferredSize(new Dimension(750, 150));
105
106     panelUtama_3002.add(areaHasil_3002, BorderLayout.SOUTH);
107 }
108
109 public void displayGraph() {
110     String visual_3002 =
111         "Gedang ----- Parkiran ----- Kantin ----- Asrama\n" +
112         " |                               |                               |\n" +
113         "Rektorat ----- Perpustakaan ----- Lab Komputer ----- Klinik\n" +
114         " |                               |                               |\n" +
115         "Aula ----- FTI ----- Masjid -----+ \n";
116
117     areaGraph_3002.setText(visual_3002);
118
119     areaHasil_3002.setText(
120         "Graph berhasil ditampilkan.\n" +
121         "Jumlah Node : " + graph_3002.size() + "\n" +
122         "Jumlah Edge : 16\n"
123     );
124 }
125
126 public void BFS() {
127     String awal_3002 = comboAwal_3002.getSelectedItem().toString();
128     String tujuan_3002 = comboTujuan_3002.getSelectedItem().toString();
129
130     Queue<String> queue_3002 = new LinkedList<>();
131     Set<String> visited_3002 = new LinkedHashSet<>();
132     Map<String, String> parent_3002 = new HashMap<>();
133
134     queue_3002.add(awal_3002);
135     visited_3002.add(awal_3002);

```

```

136 while (!queue_3002.isEmpty()) {
137     String current_3002 = queue_3002.poll();
138
139     if (current_3002.equals(tujuan_3002)) {
140         break;
141     }
142
143     for (String tetangga_3002 : graph_3002.get(current_3002)) {
144         if (!visited_3002.contains(tetangga_3002)) {
145             visited_3002.add(tetangga_3002);
146             parent_3002.put(tetangga_3002, current_3002);
147             queue_3002.add(tetangga_3002);
148         }
149     }
150 }
151
152 displayPath("BFS", awal_3002, tujuan_3002, visited_3002, parent_3002);
153
154 }
155
156 public void DFS() {
157     String awal_3002 = comboAwal_3002.getSelectedItem().toString();
158     String tujuan_3002 = comboTujuan_3002.getSelectedItem().toString();
159
160     Set<String> visited_3002 = new LinkedHashSet<>();
161     Map<String, String> parent_3002 = new HashMap<>();
162
163     dfsHelper_3002(awal_3002, tujuan_3002, visited_3002, parent_3002);
164
165     displayPath("DFS", awal_3002, tujuan_3002, visited_3002, parent_3002);
166 }
167
168 private boolean dfsHelper_3002(String current_3002, String tujuan_3002,
169                               Set<String> visited_3002,
170                               Map<String, String> parent_3002) {
171     visited_3002.add(current_3002);
172
173     if (current_3002.equals(tujuan_3002)) {
174         return true;
175     }
176
177     for (String tetangga_3002 : graph_3002.get(current_3002)) {
178         if (!visited_3002.contains(tetangga_3002)) {
179             parent_3002.put(tetangga_3002, current_3002);
180

```

```

182     if (dfsHelper_3002(tetangga_3002, tujuan_3002, visited_3002, parent_3002)) {
183         return true;
184     }
185 }
186 }
187 return false;
188 }
189 }
190
191 public void displayPath(String metode_3002, String awal_3002, String tujuan_3002,
192     Set<String> visited_3002,
193     Map<String, String> parent_3002) {
194     ArrayList<String> jalur_3002 = new ArrayList<>();
195     String current_3002 = tujuan_3002;
196     while (current_3002 != null) {
197         jalur_3002.add(current_3002);
198         if (current_3002.equals(awal_3002)) {
199             break;
200         }
201         current_3002 = parent_3002.get(current_3002);
202     }
203     Collections.reverse(jalur_3002);
204     arealHasil_3002.setText(
205         "Metode Pencarian      : " + metode_3002 + "\n" +
206         "Lokasi Awal              : " + awal_3002 + "\n" +
207         "Lokasi Tujuan             : " + tujuan_3002 + "\n" +
208         "Jalur Ditemukan           : " + String.join(" -> ", jalur_3002) + "\n" +
209         "Node Dikunjungi           : " + String.join(" -> ", visited_3002) + "\n" +
210         "Jumlah Node Dikunjungi    : " + visited_3002.size()
211     );
212 }
213
214 public void displayPath() {
215     arealHasil_3002.append("\nJalur sudah ditampilkan.");
216 }
217
218 public void resetGraph() {
219     displayGraph();
220     comboAwal_3002.setSelectedIndex(0);
221 }

```

```

222     comboTujuan_3002.setSelectedIndex(0);
223 }
224
225 public static void main(String[] args) {
226     SwingUtilities.invokeLater(() -> {
227         new PetaLokasi_2511533002().setVisible(true);
228     });
229 }
230 }

```

3.1.2 Output

1. BFS

Pencarian Jalur BFS dan DFS - 2511533002

PENCARIAN JALUR PETA KAMPUS

Lokasi Awal : Lokasi Tujuan : [BFS] [DFS] [RESET]

VISUALISASI GRAPH

```

Gerbang  ----- Parkiran ----- Kantin ----- Asrama
|         |         |         |
Rektorat ----- Perpus ----- Lab Komputer ---- Klinik
|         |         |         |
Aula ----- FTI ----- Masjid -----+
        
```

HASIL PENCARIAN

```

Metode Pencarian      : BFS
Lokasi Awal          : Gerbang
Lokasi Tujuan         : FTI
Jalur Ditemukan       : Gerbang -> Parkiran -> Perpus -> FTI
Node Dikunjungi       : Gerbang -> Parkiran -> Rektorat -> Kantin -> Perpus -> Aula -> A
Jumlah Node Dikunjungi : 11
        
```

2. DFS



3.1.3 Deskripsi Peta

Peta yang dibuat pada program ini merupakan peta sederhana lingkungan kampus. Setiap lokasi pada kampus direpresentasikan sebagai node, sedangkan jalan atau hubungan antar lokasi direpresentasikan sebagai edge. Program ini menggunakan struktur data Graph tidak berarah, sehingga setiap lokasi dapat terhubung dua arah.

Lokasi yang digunakan pada peta kampus terdiri dari beberapa tempat seperti Gerbang, Parkiran, Rektorat, Perpustakaan, Kantin, Lab Komputer, FTI, Aula, Asrama, Klinik, dan Masjid. Program ini dibuat untuk mencari jalur dari lokasi awal menuju lokasi tujuan menggunakan algoritma BFS dan DFS.

3.1.4 Jumlah Node dan Edge

1. Node :

- a. Gerbang
- b. Parkiran
- c. Rektorat
- d. Kantin
- e. Perpustakaan
- f. Aula
- g. Lab Komputer
- h. FTI
- i. Asrama
- j. Klinik
- k. Masjid

2. Edge :

- a. Gerbang - Parkiran
- b. Gerbang - Rektorat

- c. Parkiran - Kantin
- d. Parkiran - Perpus
- e. Rektorat - Perpus
- f. Rektorat - Aula
- g. Perpus - Lab Komputer
- h. Perpus - FTI
- i. Kantin – Asrama
- j. Kantin - Lab Komputer
- k. Lab Komputer - Klinik
- l. Lab Komputer - Masjid
- m. FTI - Masjid
- n. Aula - FTI
- o. Asrama - Klinik
- p. Masjid – Klinik

3.1.5 Hasil BFS

Pengujian dari Gerbang dan lokasi tujuan adalah Klinik.

- **Jalur yang ditemukan :** Gerbang -> Parkiran -> Kantin -> Asrama -> Klinik
- **Node yang dikunjungi :** Gerbang -> Parkiran -> Rektorat -> Kantin -> Perpus -> Aula -> Asrama -> Lab Komputer -> FTI -> Klinik -> Masjid
- **Jumlah node yang dikunjungi :** 11 node.

3.1.6 Hasil DFS

Pengujian dari Gerbang dan lokasi tujuan adalah Klinik.

- **Jalur yang ditemukan :** Gerbang -> Parkiran -> Kantin -> Asrama -> Klinik
- **Node yang dikunjungi :** Gerbang -> Parkiran -> Kantin -> Asrama -> Klinik
- **Jumlah node yang dikunjungi :** 5 node

BAB III

KESIMPULAN

Berdasarkan praktikum yang telah dilakukan, Node berfungsi sebagai bagian dasar dalam pembentukan binary tree karena menyimpan data serta referensi ke anak kiri dan anak kanan. BTree digunakan untuk mengelola pohon biner, seperti mengatur root, menghitung jumlah node, melakukan pencarian, dan menampilkan traversal. Sementara itu, BTreeDriver berperan sebagai class utama untuk menjalankan program, membuat node, menghubungkan node, serta menampilkan hasil traversal tree.

Program GraphTraversal digunakan untuk memahami struktur data graph dan cara menelusuri simpul-simpul yang saling terhubung. Algoritma DFS menelusuri graph secara mendalam terlebih dahulu, sedangkan BFS menelusuri graph secara melebar berdasarkan level. Dengan mempelajari Node, BTree, BTreeDriver, dan GraphTraversal, mahasiswa dapat memahami penerapan struktur data tree dan graph dalam Java serta mengetahui perbedaan cara kerja masing-masing metode penelusuran.

